

Procesadores de Lenguaje

PRÁCTICA FINAL - TERCERA PARTE



Grado en Ingeniería Informática

Fecha de entrega: 19 / 04 / 2026

Nombre	NIA	Grupo	Equipo
Raoul Vlad Ivaszuk Ivaszuk	100508621	85	509
Jorge Lopez Alonso	100495876	85	509

Índice

1. Especificaciones Frontend	1
1. Especificación 1: Variables globales	1
A) Definición	1
B) Definición con inicialización	1
C) Encadenación	1
2. Especificación 2: Función Main	3
3. Especificación 3: Impresión de cadenas	5
4. Especificación 4: Impresión de expresiones y cadenas	6
A) Versión simplificada	6
B) Versión ampliada	7
5. Especificación 5: Operadores, precedencia y asociatividad	8
6. Especificación 6: Estructura de control WHILE	10
7. Especificación 7: Estructura de control IF	11
8. Especificación 8: Variables locales	12
9. Especificación 9: Estructura For	14
*. Corrección de declaración de variables	16
10. Especificación 10: Estructura Switch/case	18
11. Especificación 11: Funciones	21
A) Definición inicial	21
B) Argumentos	21
C) Retornos	22
D) Función como procedimiento	24
12. Especificación 12: Implementación de vectores	25
A) Declaración de vectores	25
B) Operando (lectura de posiciones de vector)	26
C) Receptor (asignación en posiciones de vector)	26
D) Vectores como variables globales/locales y restricción en parámetros	26
2. Especificaciones Backend	27
1. Especificación 1: Variables Globales	27
2. Especificación 2: Funciones genéricas	27
3. Especificación 3: Imprimir cadenas	28
4. Especificación 4: Impresión de expresiones enteras	28
5. Especificación 5: Asignación de variables (setf)	29
6. Especificación 6: Estructura de control While	29
7. Especificación 8: Operadores, operandos y variables	30
Nota sobre la traducción de progn	31
8. Nota sobre la llamada a main y la directiva //@	31

1. Especificaciones Frontend

1. Especificación 1: Variables globales

A) Definición

Para el desarrollo de esta punto lo que hemos hecho ha sido añadir a las producciones de sentencia lo siguiente:

```
C/C++
sentencia: IDENTIF '=' expresion { sprintf (temp, "(setq %s %s)", $1.code,
$3.code) ;
                                $$$.code = gen_code (temp) ; }
      | '@' expresion { sprintf (temp, "(print %s)", $2.code) ;
                        $$$.code = gen_code (temp) ; }
      | INTEGER IDENTIF { sprintf (temp, "(setq %s 0)", $2.code);
                        $$$.code = gen_code(temp); }
;

```

De esta forma reconocemos los tokens correspondientes a *int* y el nombre de la variable poniendo como valor por defecto el 0.

B) Definición con inicialización

Para este paso además hemos creado un nuevo NT llamado *r_integer* que recoja la posibilidad de asignar o no un valor a la declaración.

```
C/C++
r_integer:
    { sprintf (temp, "0");
      $$$.code = gen_code(temp);} // lambda
  | '=' NUMBER { sprintf (temp, "%s", int_to_string($2.value));
                $$$.code = gen_code(temp);}
;

```

C) Encadenación

Y por último, para permitir encadenamiento, hemos extraído la producción de estas definiciones a un nuevo NT llamado *integer* y sobre este hemos añadido la recursividad necesaria.

```
C/C++
sentencia:
    ...
    | integer { sprintf (temp, "%s", $1.code);
               $$$.code = gen_code(temp);}
;
integer:
    ...

```

```
| INTEGER IDENTIF r_integer ',' integer { sprintf (temp, "(setq %s %s) %s",  
$2.code, $3.code, $5.code);  
                                          $$code = gen_code(temp); }  
;
```

2. Especificación 2: Función Main

En la especificación 2 hemos reorganizado la gramática para que el programa tenga una estructura clara y obligatoria: primero declaraciones globales y después la función main.

También hemos movido las sentencias al bloque de main, generado su traducción en Lisp como definición de función, y añadido la llamada final para ejecutar el programa traducido.

Además, hemos ajustado la salida de impresión para que use princ, en línea con el formato esperado en el enunciado.

C/C++

```
axioma:  dec_var def_func  { if (strlen ($1.code) > 0) {
                                printf ("%s\n", $1.code) ;
                                }
                                printf ("%s\n", $2.code) ;
                                printf ("//@ (main)\n")
                            }
;

dec_var:  { $$code = gen_code ("") ; }
          |  dec_var integer ';' {
                                if (strlen ($1.code) > 0) {
                                    sprintf (temp, "%s\n%s",
$1.code, $2.code) ;

                                } else {
                                    sprintf (temp, "%s",
$2.code) ;

                                }
                                $$code = gen_code (temp) ;
                            }
;

def_func:  MAIN '(' ')' '{' bloque_sentencias '}' {
                                sprintf (temp, "(defun main
()\n%s\n)", $5.code) ;

                                $$code = gen_code (temp) ;
                            }
;
.
.
.
sentencia:  IDENTIF '=' expresion  { sprintf (temp, "(setq %s %s)", $1.code,
$3.code) ;

                                $$code = gen_code (temp) ; }
          |  '@' expresion  { sprintf (temp, "(princ %s)", $2.code) ;
                                $$code = gen_code (temp) ;
                            }
```

```
}  
;
```

3. Especificación 3: Impresión de cadenas

En esta especificación sencillamente se ha añadido en la gramática en la regla de sentencia la opción de imprimir la cadena.

C/C++

sentencia:

```
...  
| PUTS '(' STRING ')'  
{ sprintf (temp, "(print \"%s\")", $3.code) ;  
  $$code = gen_code (temp) ; }
```

4. Especificación 4: Impresión de expresiones y cadenas

A) Versión simplificada

En el punto 4A hemos hecho una refactorización concreta de la parte de impresión para ajustarnos al enunciado sin adelantar funcionalidades de 4B. El cambio se concentra en trad1.y: incorporamos `printf` como palabra reservada (token propio), sustituimos la forma anterior de impresión por la producción `printf(cadena_formato, elem1)` y añadimos una no terminal auxiliar para tipar correctamente ese segundo argumento.

A nivel semántico, hemos seguido exactamente la simplificación pedida para esta fase: la cadena de formato se reconoce sólo para validar la sintaxis, pero no se interpreta. La traducción generada es una única instrucción Lisp con `princ` sobre `elem1`. Esto nos permite mantener una traducción limpia, determinista y fácil de depurar, dejando la semántica de formatos y múltiples argumentos para el siguiente subapartado.

```
C/C++
%token PRINTF
...
sentencia:  IDENTIF '=' expresion { sprintf (temp, "(setq %s %s)", $1.code,
$3.code) ;
                                $$code = gen_code (temp) ; }
          | PUTS '(' STRING ')' { sprintf (temp, "(print \"%s\")",
$3.code) ;
                                $$code = gen_code (temp) ; }
          | PRINTF '(' STRING ',' elem_print ')' { sprintf (temp, "(princ
%s)", $5.code) ;
                                $$code = gen_code (temp) ; }
;

elem_print:  expresion { $$ = $1 ; }
          | STRING { sprintf (temp, "\"%s\"", $1.code) ;
                    $$code = gen_code (temp) ; }
;
...
t_keyword keywords [] = { // define las palabras reservadas y los
    "main",           MAIN,           // y los token asociados
    "int",             INTEGER,
    "puts",            PUTS,
    "printf",          PRINTF,
    NULL,              0              // para marcar el fin de la tabla
} ;
```

B) Versión ampliada

En el punto 4B ampliamos la sentencia `printf` para aceptar varios argumentos de impresión, no solo uno. Para ello, en `trad1.y` cambiamos la regla a `PRINTF '(' STRING ' ' lista_print ')'` y definimos una lista recursiva (`lista_print`) con caso base y caso con coma, de forma que el parser procese correctamente secuencias de elementos.

A nivel semántico, cada elemento de la lista se traduce a una instrucción (`princ ...`), concatenada en el mismo orden en que aparece en C. Así, mantenemos la simplificación pedida en este apartado: la cadena de formato se valida sintácticamente, pero no se interpreta todavía. Con esto, `printf` con múltiples elementos queda implementado y funcionando según el enunciado.

```
C/C++
sentencia:  IDENTIF '=' expresion  { sprintf (temp, "(setq %s %s)", $1.code,
$3.code) ;
                                $$$.code = gen_code (temp) ; }
      | PUTS '(' STRING ')' { sprintf (temp, "(print \"%s\")", $3.code);
                                $$$.code = gen_code (temp) ; }
      | PRINTF '(' STRING ' ' lista_print ')' { $$ = $5 ; }
;
elem_print:  expresion { $$ = $1 ; }
      | STRING { sprintf (temp, "\"%s\"", $1.code) ;
                  $$$.code = gen_code (temp) ; }
;

lista_print:  elem_print { sprintf (temp, "(princ %s)",
$1.code) ;
                  $$$.code = gen_code (temp) ; }
      | lista_print ' ' elem_print { sprintf (temp, "%s\n(princ %s)",
$1.code, $3.code) ;
                  $$$.code = gen_code (temp) ; }
;
```

5. Especificación 5: Operadores, precedencia y asociatividad

En el punto 5 hemos ampliado el traductor para soportar operadores lógicos, de comparación y módulo, manteniendo una jerarquía de precedencia coherente con C y generando su equivalente en Lisp.

A nivel de gramática, incorporamos los operadores

- Lógicos: &&, ||, !
- Comparación: ==, !=, <, <=, >, >=
- Aritmético adicional: %

Además, reorganizamos la precedencia y asociatividad para evitar ambigüedades en expresiones mixtas (por ejemplo, combinación de comparaciones, lógicos y aritméticos). Con esto conseguimos que expresiones complejas se analicen de forma consistente con el comportamiento esperado del lenguaje origen.

En la traducción semántica, definimos el mapeo directo a Lisp:

- && -> (and ...)
- || -> (or ...)
- ! -> (not ...)
- == -> (= ...)
- != -> (/= ...)
- % -> (mod ...)
- Comparaciones <, <=, >, >= con su forma Lisp equivalente.

Por último, también actualizamos la tabla de palabras reservadas para reconocer los operadores dobles en el analizador léxico.

C/C++

```

expresion: termino { $$ = $1 ; }
          | expresion OR expresion { sprintf (temp, "(or %s %s)",
$1.code, $3.code) ;
                                   $$code = gen_code (temp) ; }
          | expresion AND expresion { sprintf (temp, "(and %s %s)",
$1.code, $3.code) ;
                                   $$code = gen_code (temp) ; }
          | expresion EQ expresion { sprintf (temp, "(= %s %s)", $1.code,
$3.code) ;
                                   $$code = gen_code (temp) ; }
          | expresion NE expresion { sprintf (temp, "(/= %s %s)",
$1.code, $3.code) ;
                                   $$code = gen_code (temp) ; }

```

```

        | expression '<' expression { sprintf (temp, "< %s %s)",
$1.code, $3.code) ;
                                $.code = gen_code (temp) ; }
        | expression LE expression { sprintf (temp, "<= %s %s)",
$1.code, $3.code) ;
                                $.code = gen_code (temp) ; }
        | expression '>' expression { sprintf (temp, "> %s %s)",
$1.code, $3.code) ;
                                $.code = gen_code (temp) ; }
        | expression GE expression { sprintf (temp, ">= %s %s)",
$1.code, $3.code) ;
                                $.code = gen_code (temp) ; }
        ...
                                $.code = gen_code (temp) ; }
        | expression '/' expression { sprintf (temp, "/ %s %s)",
$1.code, $3.code) ;
                                $.code = gen_code (temp) ; }
        | expression '%' expression { sprintf (temp, "(mod %s %s)",
$1.code, $3.code) ;
                                $.code = gen_code (temp) ; }
        ;

termino:
...
        | '!' operando %prec UNARY_SIGN { sprintf (temp, "(not
%s)", $2.code) ;
                                $.code = gen_code
(temp) ; }
        ;

```

6. Especificación 6: Estructura de control WHILE

En la especificación 6 incorporamos el soporte completo del bucle while dentro del traductor, tanto en análisis sintáctico como en generación de código. Definimos la nueva construcción *while (expr) { codigo }* como sentencia válida del lenguaje y la integramos en los bloques sin exigir ';' al final de la propia sentencia *while*, respetando así el comportamiento esperado en C.

A nivel semántico, establecimos su traducción directa a Lisp con la forma *(loop while <condicion> do <cuerpo>)*, manteniendo el orden y la estructura del bloque interno. Además, actualizamos el análisis léxico para que *while* se reconozca como palabra reservada y no como identificador.

```
C/C++
bloque_sentencias:
...
    |   bloque_sentencias sentencia_while {
                                if (strlen ($1.code) > 0) {
                                    sprintf (temp, "%s\n%s",
$1.code, $2.code) ;
                                } else {
                                    sprintf (temp, "%s",
$2.code) ;
                                }
                                $$code = gen_code (temp) ;
                                }
;
sentencia_while:
    WHILE '(' expresion ')' '{' bloque_sentencias '}' {
                                if (strlen ($6.code) > 0) {
                                    sprintf (temp, "(loop while %s
do\n%s)", $3.code, $6.code) ;
                                } else {
                                    sprintf (temp, "(loop while %s
do)", $3.code) ;
                                }
                                $$code = gen_code (temp) ;
                                }
;
;
```

7. Especificación 7: Estructura de control IF

En esta especificación se incorpora la estructura de control *IF* y para ello lo primero que vamos a hacer es definir los Tokens *IF* y *ELSE* y también vamos a añadir sus palabras reservadas correspondientes a la tabla de *keywords*. Además también hemos definido la nueva construcción *st_if* que da la posibilidad de generar sentencias *if-then* e *if-then-else* y una nueva producción en el bloque de sentencias para dar la posibilidad de generar estas estructuras.

Una vez tenemos la gramática estructurada, hemos pasado al analizador sintáctico. Como en C, tenemos la posibilidad de tener más de una sentencia por instancia de *if* pero en LISP eso no es posible, por ello, en esta sección hacemos uso de la función *progn* que nos permite anidar estructuras *if* y acumular múltiples sentencias.

```
C/C++
%token IF
%token ELSE

t_keyword keywords [] = {
    ...
    "if",          IF,
    "else",        ELSE,
    NULL,          0 };

st_if:
    IF '(' expresion ')' '{' bq_sent '}'          { sprintf (temp, "(if
%s\n(progn\n%s\n)\n)", $3.code, $6.code) ;
                                                    $$code = gen_code (temp) ; }

|   IF '(' expresion ')' '{' bq_sent '}' ELSE '{' bq_sent '}'
        {      sprintf          (temp,          "(if
%s\n(progn\n%s\n)\n(progn\n%s\n)\n)",          $3.code,
$6.code, $10.code) ;
        $$code = gen_code (temp) ; }

;

bq_sent:
    ...
    |   bq_sent st_if
        { if (strlen ($1.code) > 0) {
sprintf (temp, "%s\n%s", $1.code, $2.code);
        } else {
sprintf (temp, "%s", $2.code) ;
        }
        $$code = gen_code (temp) ; }

;
```

8. Especificación 8: Variables locales

Para cumplir con esta especificación lo primero que hemos hecho ha sido crear una tabla y sus funciones auxiliares para poder gestionar y almacenar las variables locales en ella y poder identificarlas. Cada vez que se quiere declarar una variable en la función main esta se almacena en esta tabla usando la función `add_local()`, posteriormente, cuando esta desee ser usada se recuperará su nombre con la función `get_var_name()`.

También, en cuanto a la gramática, se ha añadido una producción en el *NT dec_func* que permite declarar variables locales antes del bloque de sentencias. Además también se han añadido nuevas construcciones posibles para estas variables locales iguales a las declaraciones de las variables globales pero con acciones semánticas diferentes.

En la parte semántica, estas declaraciones se siguen haciendo con la función `setq` con la excepción de que el nombre de la variable se concatena con el nombre de la función a la que pertenece (`main_`) además también se han cambiado las asignaciones, ahora usan la función `setf` para diferenciarlas de estas declaraciones.

```
C/C++
dec_var_local:
    { $$code = gen_code ("") ; } // lambda
|   dec_var_local integer_local ';' { if (strlen ($1.code) > 0) {
    sprintf (temp, "%s\n%s", $1.code, $2.code) ;
    } else {
    sprintf (temp, "%s", $2.code) ;
    }
    $$code = gen_code (temp) ; }
;

integer_local:
    INTEGER IDENTIF r_integer { add_local($2.code); // La guardamos en la
    tabla
    sprintf (temp, "(setq main_%s %s)", $2.code,
    $3.code);
    $$code = gen_code(temp); }
|   INTEGER IDENTIF r_integer ',' integer_local
    { add_local($2.code);
    sprintf (temp, "(setq main_%s %s)\n%s",
    $2.code, $3.code, $5.code);
    $$code = gen_code(temp); }
;

def_func:
    MAIN '(' ')' '{' dec_var_local bq_sent '}' { sprintf (temp, "(defun main
    ())\n%s\n%s\n)", $5.code, $6.code) ;
    $$code = gen_code (temp) ; }
;
```

```
    sentencia:      IDENTIF '=' expresion    { sprintf (temp, "(setf %s %s)",
get_var_name($1.code), $3.code) ;
                                                    $$ .code = gen_code (temp) ; }

    ...

    char  tabla_locales[100][256]; // Tabla para guardar hasta 100 variables
locales
    int num_locales = 0;

    void add_local(char* nombre) {
        strcpy(tabla_locales[num_locales++], nombre);
    }

    int es_local(char* nombre) {
        for(int i = 0; i < num_locales; i++) {
            if(strcmp(tabla_locales[i], nombre) == 0) return 1;
        }
        return 0; // No está en la tabla, por lo que es global
    }

    char* get_var_name(char* id) {
        if (es_local(id)) {
            char temp_name[256];
            sprintf(temp_name, "main_%s", id); // Le añade el prefijo
            return gen_code(temp_name);
        }
        return id; // Si es global, devuelve el nombre tal cual
    }
}
```

9. Especificación 9: Estructura For

En la especificación 9 incorporamos el soporte del bucle for del subconjunto de C, restringiéndolo a una forma canónica para simplificar su traducción y evitar ambigüedades en la gramática. La estructura que implementamos fue de la forma `for (inicializa; expr; inc/dec) { codigo }`, donde la inicialización se interpreta como una asignación, la condición como una expresión booleana o relacional, y la parte de incremento o decremento como una llamada a los procedimientos `INC(x)` o `DEC(x)`.

Para traducirlo a Lisp, reutilizamos la estructura del `while`, adaptando el comportamiento del incremento para que se ejecute al final de cada iteración. De este modo, la semántica del bucle se preserva de forma equivalente, pero expresada con la construcción `loop while xxx do xxx`. También limitamos el uso de `INC` y `DEC` a procedimientos explícitos, evitando incorporar operadores tipo `++` o `--` en el traductor, tal y como indicaba el enunciado.

C/C++

```
...
%token FOR          // identifica el bucle for
%token INC          // macro para incremento
%token DEC
...
bq_sent:
    ...
    | bq_sent st_for { if (strlen ($1.code) > 0) {
                                sprintf (temp, "%s\n%s",
                                $1.code, $2.code) ;
                                } else {
                                sprintf (temp, "%s", $2.code) ;
                                }
                                $$code = gen_code (temp) ;
                                }
;

...
op_inc_dec: INC '(' IDENTIF ')' { sprintf (temp, "(setf %s (+ %s 1))",
get_var_name($3.code), get_var_name($3.code)) ;
                                $$code = gen_code (temp) ; }
    | DEC '(' IDENTIF ')' { sprintf (temp, "(setf %s (- %s 1))",
get_var_name($3.code), get_var_name($3.code)) ;
                                $$code = gen_code (temp) ; }
;

...
st_for: FOR '(' sentencia ';' expresion ';' op_inc_dec ')' '{' bq_sent '}'
        {
            if (strlen ($10.code) > 0) {
```

```

// Si hay código en el bloque, el
incremento va al final del bloque
    sprintf (temp, "%s\n(loop while %s
do\n%s\n%s)", $3.code, $5.code, $10.code, $7.code) ;
    } else {
// Si el bloque está vacío, solo se
ejecuta el incremento dentro del bucle
    sprintf (temp, "%s\n(loop while %s
do\n%s)", $3.code, $5.code, $7.code) ;
    }
    $$code = gen_code (temp) ;
}
;
...
t_keyword keywords [] = { // define las palabras reservadas y los
    "main",      MAIN,      // y los token asociados
    "int",       INTEGER,
    "while",     WHILE,
    "for",       FOR,
    "inc",       INC,
    "dec",       DEC,
    ...

```

*. Corrección de declaración de variables

Antes de seguir con las especificaciones pertinentes, nos dimos cuenta de que cometimos un ligero error. Corregimos la forma en que se generan las declaraciones `int`, tanto globales como locales. Detectamos que, al manejar listas de variables separadas por comas, el traductor estaba repitiendo el tipo en cada nombre, produciendo salidas del estilo `int a, int b, int c`, cuando lo correcto era declarar el tipo una sola vez y generar después cada variable de forma individual. Para resolverlo, reestructuramos la gramática de declaraciones para que el `int` actúe como cabecera común de la lista, y cada identificador se traduzca por separado con su posible inicialización. Con esto conseguimos una salida más correcta y coherente con la sintaxis esperada del lenguaje.

```
C/C++
integer_local: INTEGER lista_integer_local { $$ = $2 ; }
;
lista_integer: IDENTIF r_integer { add_local($1.code);
                                sprintf (temp, "(setq main_%s %s)", $1.code,
                                $2.code);
                                $$$.code = gen_code(temp); }
    | IDENTIF r_integer ',' lista_integer_local { add_local($1.code);
                                                sprintf
(temp, "(setq main_%s %s)\n%s", $1.code, $2.code, $4.code);
                                                $$$.code =
gen_code(temp); }
;
...

integer: INTEGER lista_integer { $$ = $2 ; }
;

lista_integer: IDENTIF r_integer { sprintf (temp, "(setq %s %s)", $1.code,
$2.code);
                                $$$.code = gen_code(temp); }
    | IDENTIF r_integer ',' lista_integer_global { sprintf (temp, "(setq %s
%s)\n%s", $1.code, $2.code, $4.code);
                                $$$.code =
gen_code(temp); }
;
...
int yyerror (mensaje)
...
    return 0 ;
}
...
t_keyword keywords [] = { // define las palabras reservadas y los
```

```
{ "main",      MAIN },          // y los token asociados
{ "int",       INTEGER },
{ "while",     WHILE },
{ "for",       FOR },
{ "inc",       INC },
{ "dec",       DEC },
{ "puts",      PUTS },
{ "printf",    PRINTF },
{ "&&",        AND },
{ "||",        OR },
{ "==" ,       EQ },
{ "!=" ,       NE },
{ "<=" ,       LE },
{ ">=" ,       GE },
{ "if",        IF },
{ "else",      ELSE },
{ NULL,        0 }              // para marcar el fin de la tabla
} ;
```

10. Especificación 10: Estructura Switch/case

En la especificación 10 añadimos el soporte de *switch/case/default/break*, de nuevo restringiendo la sintaxis a una versión básica y controlada para poder traducirla de forma sencilla a Lisp. La construcción se integró como una nueva sentencia dentro de los bloques del lenguaje, permitiendo varios *case* y un *default* opcional, todos ellos terminados con *break*.

La traducción semántica se realizó usando la estructura *case* de Lisp, donde cada *case* de C se corresponde con una rama evaluada sobre la expresión principal del *switch*, y *default* se traduce como *otherwise*. Con esta incorporación conseguimos representar de manera natural la selección múltiple de casos, manteniendo el orden de evaluación esperado y respetando la estructura del programa original.

```
C/C++
%token SWITCH CASE DEFAULT BREAK
...
bq_sent:
...
    | bq_sent st_switch { if (strlen ($1.code) > 0) {
                                sprintf (temp, "%s\n%s",
$1.code, $2.code) ;
                                } else {
                                    sprintf (temp, "%s", $2.code) ;
                                }
                                $$code = gen_code (temp) ;
                                }
;
...
st_switch:      SWITCH '(' expresion ')' '{' lista_cases opt_default '}' {
if (strlen ($6.code) > 0) {
    sprintf (temp, "(case %s\n%s\n%s\n)", $3.code, $6.code, $7.code) ;
} else {
    sprintf (temp, "(case %s\n%s\n)", $3.code, $7.code) ;
}
    $$code = gen_code (temp) ;
}
;

lista_cases:    case_item {
    $$ = $1 ; }
```

```

        | lista_cases case_item                                {
sprintf (temp, "%s\n%s", $1.code, $2.code) ;

$$$.code = gen_code (temp) ; }
        ;

case_item:          CASE NUMBER ':' bq_sent BREAK ';'        {

if (strlen ($4.code) > 0) {

sprintf (temp, "(%d\n%s\n)", $2.value, $4.code) ;

} else {

sprintf (temp, "(%d)", $2.value) ;

}

$$$.code = gen_code (temp) ;

}

        ;

opt_default:                                                {
$$$.code = gen_code ("") ; }
        | DEFAULT ':' bq_sent BREAK ';'                    {

if (strlen ($3.code) > 0) {

sprintf (temp, "(otherwise\n%s\n)", $3.code) ;

} else {

sprintf (temp, "(otherwise)") ;

}

$$$.code = gen_code (temp) ;

}

;
...
t_keyword keywords [] = {
...
{ "switch",      SWITCH },
{ "case",        CASE },
{ "default",     DEFAULT },
{ "break",       BREAK },

```

...

11. Especificación 11: Funciones

A) Definición inicial

Como aproximación inicial, se amplió la gramática para soportar la definición de múltiples funciones además de *main*. En lenguaje C, si se omite el tipo de retorno explícito, se asume implícitamente *int*, lo que nos permitió simplificar el diseño evitando conflictos con la declaración de variables (<tipo> <identificador>).

Además para evitar el problema de los prototipos en el NT se ha obligado que estas nuevas funciones se declaren todas antes del bloque *main*. Con todo esto, las traducciones que se realizan son traducciones directas de c al lenguaje Lisp.

```
C/C++
def_func:          def_otras def_main      { sprintf (temp, "%s\n%s", $1.code,
$2.code) ;

                                     $$code = gen_code (temp) ; }

;

def_main:          MAIN '(' ')' '{' dec_var_local bq_sent '{'
{ sprintf (temp, "(defun main ()\n%s\n%s\n)", $5.code, $6.code) ;
                                     $$code = gen_code (temp) ; }

;

def_otras:          { $$code = gen_code ("") ; } // lambda
| IDENTIF '(' ')' '{' dec_var_local bq_sent '{' def_otras
{ if (strlen ($8.code) > 0) {
  sprintf (temp, "%(defun %s ()\n%s\n%s\n)\n%s", $1.code,
$5.code, $6.code, $8.code) ;
} else {
  sprintf (temp, "(defun %s ()\n%s\n%s\n)", $1.code, $5.code,
$6.code) ; }
  $$code = gen_code (temp) ; }

;
```

B) Argumentos

Posteriormente, se extendió la cabecera de las funciones para admitir cero, uno o múltiples parámetros mediante las reglas *lista_param_def* (para la declaración) y *lista_arg* (para las llamadas).

```
C/C++
def_otras:
...
| IDENTIF '(' lista_param_def ')' '{' dec_var_local bq_sent '{' def_otras
```

```

{ if (strlen ($9.code) > 0) {
sprintf (temp, "(defun %s (%s)\n%s\n%s)\n%s", $1.code, $3.code, $6.code,
$7.code, $9.code) ;
...
lista_param_def:          { $$ .code = gen_code ("") ; } // lambda
| param_def_nonempty      { $$ = $1 ; }
;

param_def_nonempty: INTEGER IDENTIF { $$ .code = gen_code ($2.code) ; }
| INTEGER IDENTIF ',' param_def_nonempty
{ sprintf (temp, "%s %s", $2.code, $4.code) ;
  $$ .code = gen_code (temp) ; }
;

lista_arg:                { $$ .code = gen_code ("") ; } // lambda
| arg_nonempty            { $$ = $1 ; }
;

arg_nonempty:  expresion      { $$ = $1 ; }
| expresion ',' arg_nonempty { sprintf (temp, "%s %s",
$1.code, $3.code) ;
                                $$ .code = gen_code (temp) ; }
;

operando:
...
| IDENTIF '(' lista_arg ')' { if (strlen ($3.code) > 0) {
                                sprintf (temp, "(%s %s)", $1.code, $3.code) ;
                                } else {
                                sprintf (temp, "(%s)", $1.code) ;
                                } $$ .code = gen_code (temp) ; }

```

C) Retornos

A pesar de que en Lisp los retornos se hacen solo al final de la función, para poder permitir, como en C, poder hacer retornos en medio de la función se hace uso de la directiva *return-from* de Lisp. También se ha añadido el token *RETURN* a la lista de token y su valor a las *keywords*.

Además en este punto también se ha hecho que las variables definidas en una función sean locales pertenecientes a esta función, para ello se ha definido una función en C que se encarga de modificar una variable que determina la función en la que se está trabajando. Este cambio ha afectado también a la declaración de las variables en el procedimiento principal *main*.

C/C++

```
void set_current_function(char* nombre) ;
```

```

    char current_function [256] = "" ;
def_main:      MAIN '(' ' ') { set_current_function ("main") ; } '{'
dec_var_local bq_sent '}'

                                { sprintf (temp, "(defun main ())\n%s\n%s)",
$6.code, $7.code) ;

    ...
def_otras:
    ...
    | IDENTIF { set_current_function ($1.code) ; } '(' lista_param_def ')' '{'
dec_var_local bq_sent '}' def_otras

                                { sprintf
(temp, "(defun %s (%s)\n%s\n%s)\n\n%s", $1.code, $4.code, $7.code, $8.code,
$10.code) ;
    lista_integer_local:
        IDENTIF r_integer          { add_local($1.code);
                                    sprintf (temp, "(setq %s_%s %s)",
current_function, $1.code, $2.code);

                                    $$code = gen_code(temp); }
    | IDENTIF r_integer ',' lista_integer_local { add_local($1.code);
                                                sprintf (temp, "(setq %s_%s %s)
\n%s", current_function, $1.code, $2.code, $4.code);
                                                $$code = gen_code(temp); }

    sentencia:
    ...
    | RETURN expresion          { sprintf (temp, "(return-from %s %s)",
current_function, $2.code) ;

                                $$code = gen_code (temp) ; }

void add_local(char* nombre) {
    sprintf(tabla_locales[num_locales++], "%s_%s", current_function, nombre);
}

int es_local(char* nombre) {
    char nombre_acotado[512];
    sprintf(nombre_acotado, "%s_%s", current_function, nombre);
    for(int i = 0; i < num_locales; i++) {
        if(strcmp(tabla_locales[i], nombre_acotado) == 0) return 1;
    }
    return 0; // No está en la tabla, por lo que es global
}

char* get_var_name(char* id) {
    if (es_local(id)) {
        char temp_name[512];
        sprintf(temp_name, "%s_%s", current_function, id); // Le añade el
prefijo de la función actual
        return gen_code(temp_name);
    }
}

```

```

    }
    return id; // Si es global, devuelve el nombre tal cual
}

void set_current_function(char* nombre) {
    strncpy(current_function, nombre, sizeof(current_function) - 1);
    current_function[sizeof(current_function) - 1] = '\0';
}

```

D) Función como procedimiento

Finalmente, se habilitó la capacidad de invocar funciones puramente por sus efectos colaterales (por ejemplo, funciones que solo imprimen por pantalla) ignorando su valor de retorno, una práctica permitida en C.

Para ello, en el árbol de derivación de la regla *sentencia*, se añadió una nueva producción que reconoce explícitamente *IDENTIF '(' lista_arg ')' ';' .* Esto permite que una llamada a función pueda existir como una línea de código independiente, en lugar de estar estrictamente ligada a la regla *expresion* (la cual obliga a que la función forme parte de una asignación o de un operador lógico/aritmético). Se traduce directamente generando la lista *Lisp* (*identif arg1 arg2*).

C/C++

```

| IDENTIF '(' lista_arg ')'
$3.code) ;

```

```

{ if (strlen ($3.code) > 0) {
    sprintf (temp, "(%s %s)", $1.code,
        $3.code) ;
} else {
    sprintf (temp, "(%s)", $1.code) ; }
$$code = gen_code (temp) ; }

```

12. Especificación 12: Implementación de vectores

A) Declaración de vectores

En este apartado ampliamos la gramática para permitir la declaración de vectores tanto en ámbito global como local con la sintaxis tipo C `int nombre[tam];`. A nivel de traducción, cada declaración pasó a generarse en Lisp como una inicialización del array mediante `make-array`, manteniendo el mismo esquema de nombres que ya usábamos para distinguir variables globales y locales. Con este cambio conseguimos que el traductor reconozca correctamente la reserva de memoria para vectores sin alterar el comportamiento previo de las declaraciones escalares.

```
C/C++
dec_var:
...
    | dec_var vector_decl ';' { if (strlen ($1.code) > 0) {
                                sprintf (temp,
"%s\n%s", $1.code, $2.code) ;
                                } else {
                                sprintf (temp, "%s", $2.code);
                                }
                                $$code = gen_code (temp) ;
    }
;
dec_var_local:
...
    | dec_var_local vector_decl_local ';' { if (strlen ($1.code) >
0) {
    sprintf (temp, "%s\n%s", $1.code, $2.code) ;
    } else {
    sprintf (temp, "%s", $2.code) ;
    }
    $$code = gen_code (temp) ;
    }
;
...
vector_decl:  INTEGER IDENTIF '[' NUMBER ']' { sprintf (temp, "(setq %s
(make-array %d))", $2.code, $4.value) ;
                                                $$code = gen_code (temp) ; }
;
vector_decl_local:  INTEGER IDENTIF '[' NUMBER ']' { add_local($2.code) ;
                                                        sprintf (temp, "(setq main_%s
(make-array %d))", $2.code, $4.value) ;
                                                        $$code = gen_code (temp) ; }
```

B) Operando (lectura de posiciones de vector)

Para 12.b incorporamos el acceso a posiciones de vector dentro de expresiones, de forma que construcciones como `v[i]` pudieran utilizarse como operando en asignaciones, condiciones o llamadas. La traducción semántica de este acceso se mapeó a la forma Lisp `aref`, respetando tanto el índice como el tratamiento de ámbito de la variable vectorial. Esto permitió usar elementos de vectores como cualquier otro valor dentro de la parte expresiva del lenguaje traducido.

```
C/C++
operando:
...
| IDENTIF '[' expresion ']' { sprintf (temp,
"(aref %s %s)", get_var_name($1.code), $3.code) ;
                                                                    $$ .code =
gen_code (temp) ; }
```

C) Receptor (asignación en posiciones de vector)

En este subapartado añadimos la asignación sobre elementos de vector en el lado izquierdo de la sentencia, soportando patrones como `v[i] = expr;`. La modificación principal consistió en extender las reglas de sentencia para reconocer este caso y traducirlo a una operación `setf` sobre `aref` en Lisp. Con ello cerramos el ciclo de uso básico de vectores: no solo leer posiciones, sino también escribir en ellas, manteniendo coherencia con el modelo ya implantado para asignaciones de variables normales.

```
C/C++
sentencia:
...
| IDENTIF '[' expresion ']' '=' expresion { sprintf (temp, "(setf (aref
%s %s) %s)", get_var_name($1.code), $3.code, $6.code) ;
                                                                    $$ .code = gen_code (temp) ; }
```

D) Vectores como variables globales/locales y restricción en parámetros

Para 12.d verificamos la integración completa del uso de vectores declarados en global y local, incluyendo su paso indirecto a funciones mediante elementos individuales (por ejemplo, `v[i]` como argumento escalar). Además, comprobamos la restricción exigida en el enunciado: no se contemplan parámetros vectoriales en la firma de funciones, por lo que ese patrón se rechaza sintácticamente. De esta forma, el traductor queda alineado con el alcance definido para el apartado, permitiendo uso práctico de vectores sin abrir aún la semántica de paso por referencia o arrays como parámetro formal.

2. Especificaciones Backend

1. Especificación 1: Variables Globales

En esta especificación implementamos la base del backend, es decir, la estructura mínima de traducción desde la representación intermedia en Lisp hacia un programa ejecutable en Forth. Nos centramos en dejar operativo el flujo completo de trabajo: análisis sintáctico de la entrada, generación de salida en el formato esperado y compilación del traductor sin errores.

Con esta primera fase conseguimos validar que la tubería de traducción funciona de extremo a extremo y que podemos producir código Forth correcto a nivel estructural. Esta base nos permitió afrontar las siguientes especificaciones de forma incremental, manteniendo estabilidad y facilitando la depuración.

```
C/C++
expression1:
...
    | '(' SETQ IDENTIF { printf(" variable %s ", $3.code); } number
    ')' { printf(" %s ! \n", $3.code); } // variable declaration
...
t_keyword keywords [] = {
...
    "setq",      SETQ,
    "setf",      SETF,
```

2. Especificación 2: Funciones genéricas

En la segunda especificación implementamos la traducción de la definición principal del programa, pasando de la forma Lisp (defun main () ...) a la forma Forth : main ... ;. Para ello, adaptamos las acciones semánticas de la gramática de backend para emitir el inicio de definición : main al entrar en la regla correspondiente y el cierre ; al finalizarla.

Después validamos el resultado recopilando el traductor y ejecutando la cadena completa de pruebas (C -> Lisp -> Forth -> gforth). Con esta mejora dejamos correctamente definida la entrada principal del programa en Forth, consolidando el esqueleto necesario para incorporar en las siguientes especificaciones la traducción del cuerpo y del resto de construcciones del lenguaje.

```
C/C++
expression1:
...
    | '(' DEFUN MAIN { printf (" : main ") ; }
```

```

    '(' ')' exprSeq ')' { printf ( " ;\n" ) ; }
...

```

3. Especificación 3: Imprimir cadenas

En la especificación 3 implementamos la traducción de las dos formas de impresión de cadenas del código Lisp intermedio al lenguaje Forth. El código Lisp generado por el frontend diferencia entre (print ...), que imprime la cadena seguida de un salto de línea, y (princ ...), que la imprime sin salto de línea. En Forth, la palabra ." imprime la cadena que la sigue hasta el cierre ", y cr produce el salto de línea.

Para ello añadimos dos nuevas producciones en expression1 del backend, una para cada forma de impresión. La primera emite ." <cadena>" cr al encontrar (print "..."), y la segunda emite ." <cadena>" sin cr al encontrar (princ "..."). De esta forma respetamos el comportamiento esperado en ambos casos.

```

C/C++
expression1:
...
| '(' PRINT STRING ')'
    { printf ( " .\" %s\" cr ", $3.code) ; }

| '(' PRINC STRING ')'
    { printf ( " .\" %s\" ", $3.code) ; }
...

```

4. Especificación 4: Impresión de expresiones enteras

En esta especificación completamos el soporte de princ para el caso en que su argumento es una expresión entera en lugar de una cadena literal. En Forth, la palabra . (punto) extrae el valor de la cima de la pila y lo imprime como entero, que es exactamente la semántica de (princ <expr>) cuando el argumento es numérico.

Dado que la gramática del backend usa traducción directa, el valor de la expresión ya ha sido emitido en postfija antes de reducir la regla de princ. Por tanto, basta con emitir . a continuación, sin necesidad de almacenar la traducción de la expresión. Esto mantiene el patrón de acción mínima en la reducción que venimos siguiendo.

```
C/C++
expression1:
...
| '(' PRINT STRING ')'
    { printf ( " .\" %s\" cr ", $3.code) ; }

| '(' PRINC STRING ')'
    { printf ( " .\" %s\" ", $3.code) ; }
...
```

5. Especificación 5: Asignación de variables (setf)

En esta especificación implementamos la traducción de la asignación a variables globales desde Lisp a Forth. La forma (setf <var> <expr>) del código intermedio debe traducirse a la secuencia Forth <expr> <var> !, donde ! es el operador de almacenamiento que toma de la pila una dirección (la variable) y un valor (la expresión evaluada) y los almacena.

Al igual que con princ, la traducción directa nos permite que la expresión ya haya sido emitida en postfija antes de reducir la regla setf, por lo que únicamente es necesario emitir el nombre de la variable seguido del operador de almacenamiento !. Esta regla complementa la especificación 1 (setq, declaración con valor inicial) cubriendo las asignaciones posteriores dentro del cuerpo de las funciones.

```
C/C++
expression1:
...
| '(' PRINT STRING ')'
    { printf ( " .\" %s\" cr ", $3.code) ; }

| '(' PRINC STRING ')'
    { printf ( " .\" %s\" ", $3.code) ; }
...
```

6. Especificación 6: Estructura de control While

En la especificación 6 implementamos la traducción de la estructura de bucle generada por el frontend. El código intermedio Lisp usa la forma (loop while <expr> do <cuerpo>), que el backend debe traducir a la estructura Forth BEGIN ... WHILE ... REPEAT.

La estructura Forth funciona de la siguiente manera: BEGIN marca el inicio del bucle, la expresión de condición se evalúa y deja un valor en la pila, WHILE lo consume y continúa si es verdadero o salta al final si es falso, y REPEAT transfiere la ejecución de nuevo a BEGIN. El uso de acciones de mitad de producción (mid-rule actions) nos permite emitir BEGIN antes

de que la expresión de condición sea evaluada, y WHILE justo después, manteniendo así el orden postfijo correcto sin necesidad de código diferido.

```
C/C++
expression1:    ...
    | '(' LOOP WHILE    { printf (" BEGIN\n") ; } // emite BEGIN antes de la
condicion
                expression    { printf (" WHILE\n") ; } // emite WHILE despues de
la condicion
                DO exprSeq ')' { printf (" REPEAT\n") ; } // emite REPEAT al
cerrar el bucle
    ...
```

7. Especificación 8: Operadores, operandos y variables

En la especificación 8 implementamos la traducción de todas las expresiones del subconjunto Lisp al que se ajusta el frontend: operadores aritméticos, relacionales, lógicos y unarios, así como el acceso a variables y la impresión de literales numéricos.

Todos los operadores binarios Lisp tienen notación prefija con la forma (op expr1 expr2). Gracias a la traducción directa, cuando la producción de expression se reduce, expr1 y expr2 ya han sido emitidos en la pila en orden correcto, por lo que basta con emitir el operador correspondiente en Forth al finalizar la reducción. Para el operador de desigualdad Lisp (/=), la correspondencia en Forth es = 0= (igualdad seguida de negación booleana), tal y como establece la tabla de equivalencias del enunciado. La negación unaria usa negate y (not ...) se traduce a 0= (la negación booleana en Forth).

Para el acceso a variables, el operando IDENTIF emite <nombre> @ ya que en Forth es necesario usar el operador de carga @ para leer el valor almacenado en la dirección de la variable. Los literales numéricos simplemente se emiten tal cual, ya que Forth los reconoce y los apila directamente.

```
C/C++
expression:
    operand                                { ; }
    | '(' '+' expression expression ')' { printf (" + ") ; }
    | '(' '*' expression expression ')' { printf (" * ") ; }
    | '(' '-' expression expression ')' { printf (" - ") ; }
    | '(' '/' expression expression ')' { printf (" / ") ; }
    | '(' MOD expression expression ')' { printf (" mod ") ; }
    | '(' AND expression expression ')' { printf (" and ") ; }
    | '(' OR  expression expression ')' { printf (" or ") ; }
    | '(' '=' expression expression ')' { printf (" = ") ; }
```

```

Lisp
| '(' NE expression expression ')' { printf (" = 0= ") ; } // /= en
| '(' '>' expression expression ')' { printf (" > ") ; }
| '(' '<' expression expression ')' { printf (" < ") ; }
| '(' GE expression expression ')' { printf (" >= ") ; }
| '(' LE expression expression ')' { printf (" <= ") ; }
| '(' NOT expression ')' { printf (" 0= ") ; } // negacion
booleana
| '(' '-' expression ')' { printf (" negate ") ; } // menos
unario
;

```

Nota sobre la traducción de progn

La forma (progn ...) del código Lisp intermedio, generada por el frontend para agrupar múltiples sentencias en las ramas del if, no requiere ninguna traducción explícita en Forth ya que las palabras de Forth se ejecutan secuencialmente de por sí. Por ello su regla de producción simplemente deja pasar el contenido de la secuencia sin emitir código adicional.

```

C/C++
expression1:
...
| '(' PROGN exprSeq ')' { /* sin accion: las sentencias ya se emitieron
*/ }
...

```

8. Nota sobre la llamada a main y la directiva //@

Siguiendo la especificación #11 del enunciado, el backend no imprime de forma autónoma la llamada a la función principal. Dicha llamada se gestiona exclusivamente mediante la directiva //@ (main) del código C original, que el frontend transcribe como (main) en el código Lisp intermedio, y que el backend traduce a la palabra Forth main.

Esta decisión de diseño es fundamental para que el orden del código generado sea correcto: si el backend emitiera la llamada a main al reducir el axioma, dicha llamada aparecería antes que la definición de la función, lo que causaría un error en gforth. Al diferirla a la directiva inline, se garantiza que aparece siempre al final del fichero traducido, tras todas las definiciones de funciones y variables.

```

C/C++
expression1:
...

```

```
| '(' MAIN ')'      { printf (" main\n") ; } // llamada a la funcion
principal
| '(' IDENTIF ')'  { printf (" %s \n", $2.code) ; } // llamada a funcion
sin argumentos
...

axiom: exprSeq    { ; } // NO se emite llamada a main aqui
```